

Causal Inference With Bayesian Methods

Aisha Sallah - Gina Kim - Tianhai Zhou and Roger Armand Saumure

2026-04-29

Causal Inference with Bayesian Methods

STAT 5440 - Applied Bayesian Modeling - Spring 2026

Aisha Sallah - Gina Kim - Tianhai Zhou and Roger Armand Saumure

This tutorial walks through an example of using synthetic controls for causal inference in both Frequentist and Bayesian frameworks, highlighting the advantage of using a Bayesian principles in causal inference.

I. Introduction to Causal Inference

In causal inference, the goal is to draw conclusions about cause-and-effect relationships from data. Most of what we observe in data is correlation, but correlation is not causation. When evaluating a treatment, we want to know whether it truly causes an outcome, rather than being correlated with some confounder that is the real driver. The aim is to separate the true causal effect from noise introduced by confounders — a distinction that matters across healthcare, decision science, business, and economics. The Counterfactual Problem To know whether a treatment causes an outcome — for example, whether a drug causes remission — we would ideally observe what each individual's outcome would have been with and without treatment. But only one of these can ever be observed per person: a patient either receives the drug or they don't. The unobserved outcome is the counterfactual, and estimating it is the central problem of causal inference.

This is formalized in the Neyman-Rubin potential outcomes framework. For each unit x_i , two potential outcomes exist:

$Y_0(x_i)$ — the outcome under no treatment (the control outcome) $Y_1(x_i)$ — the outcome under treatment (the treated outcome)

Only one is ever observed; the other is missing.

Estimands of Interest

Since we cannot observe both potential outcomes for the same individual, we estimate causal effects in expectation. Two estimands are of primary interest:

Conditional Average Treatment Effect (CATE) — the expected treatment effect for an individual with covariates x_i :

$$\tau(x_i) = \mathbb{E}[Y_1 - Y_0 \mid X = x_i]$$

Average Treatment Effect (ATE) — the average effect across the population:

$$\text{ATE} = \mathbb{E}[Y_1 - Y_0] = \mathbb{E}_X[\tau(X)]$$

The ATE is simply the CATE averaged over the covariate distribution.

Common Methods

A number of methods have been developed to estimate these quantities — including propensity score matching, difference-in-differences, regression discontinuity design, and instrumental variables — each with its own identification strategy and assumptions. This tutorial focuses on synthetic control, which is particularly well suited to the data used here and clearly highlights the advantages of the Bayesian framework over its frequentist counterpart. Several of the other methods could also be applied to this analysis; the appropriate choice ultimately depends on the available data, study design, methodology, and the specific question being asked.

Why a Bayesian Framework?

All of the methods above can be formulated in either a frequentist or a Bayesian framework. Frequentist implementations return point estimates with confidence intervals, while Bayesian implementations place priors on the relevant quantities and return full posterior distributions over the effects of interest. This distinction matters most for synthetic control. Frequentist inference relies on placebo tests, which are constrained by the number of donors available and do not provide a full characterization of uncertainty at every post-intervention time point. A Bayesian formulation addresses this directly: by placing priors on the donor weights and the outcome model, we obtain a full posterior distribution over the treatment effect at every time point post-intervention. This yields:

Coherent uncertainty quantification at every time point, not just an aggregate p-value
The ability to incorporate prior knowledge about plausible weights or effect sizes
Natural propagation of uncertainty from the weights through to the estimated effect

II. Dataset and Model

The Research Question

On May 12, 2021, Ohio Governor Mike DeWine announced “Vax-a-Million” — a state-run lottery in which any Ohio adult who had received at least one dose of a COVID-19 vaccine would be entered into a drawing for a \$1 million prize. The intervention was unprecedented in its use of a financial incentive at the population level, and it raises a clean causal question:

Did Ohio’s Vax-a-Million lottery cause an increase in COVID-19 vaccination rates among Ohio adults, or would those rates have risen at the same pace without it?

This question cannot be answered with a simple before/after comparison. Vaccination rates were rising rapidly across the entire country in spring 2021 — supply was expanding, the Delta variant was beginning to drive concern, and many employers and universities were pushing their populations toward vaccination. A pre/post comparison in Ohio alone would conflate the lottery’s effect with this nationwide upward trend, almost certainly overstating its impact. What we need is a credible estimate of Y_0 for Ohio in the post-treatment period: the vaccination trajectory Ohio *would have followed* had the lottery never been announced. Synthetic control gives us exactly this object.

Data Source

The vaccination data come from the CDC’s *COVID-19 Vaccinations in the United States, County* dataset, which reports daily, jurisdiction-level cumulative vaccination counts and is publicly available at:

https://data.cdc.gov/Vaccinations/COVID-19-Vaccinations-in-the-United-States-County/8xkx-amqh/about_data

For this analysis we aggregate the county-level series to the state level and follow the specification of Fuller et al. (2022), augmenting the CDC vaccination series with state-level demographic, political, and survey

covariates from public sources (the U.S. Census, the 2020 presidential election results, and the Census Bureau’s Household Pulse Survey on vaccination intent). The combined panel covers all 51 jurisdictions (50 states plus the District of Columbia) at daily frequency.

The outcome variable is `Administered_Dose1_Recip_18PlusPop_Pct` — the cumulative percentage of each state’s 18-and-over population that has received at least one vaccine dose. We focus on first-dose uptake (rather than full vaccination) because the lottery was conditioned on receipt of at least one dose, so this is the margin most directly affected by the policy.

Treated Unit, Donor Pool, and Time Window

The **treated unit** is Ohio (OH). The **treatment date** is `$T_0` = May 12, 2021 — the day the lottery was announced. The **donor pool** consists of 31 states that did not run vaccine lotteries during the period of analysis; states that subsequently introduced their own lottery programs are excluded so that the synthetic control is built only from genuinely untreated comparison units.

The analysis window runs from April 19, 2021 to early August 2021, giving roughly 23 days of pre-treatment observations and approximately 80 days of post-treatment follow-up. The pre-treatment period is what the synthetic control method uses to pin down the donor weights; the post-treatment period is where we read off the estimated effect.

Matching Variables

Beyond the daily vaccination series itself, we match Ohio to its synthetic counterpart on 13 state-level covariates spanning four broad categories:

- *Political and economic context*: 2020 Democratic two-party vote share, income per capita
- *Demographics*: percentage over 65, percentage with a bachelor’s degree or higher, racial composition (white, Black, Hispanic), population density
- *Vaccine attitudes* (from the Household Pulse Survey, measured pre-treatment): the share of respondents who said they would *definitely*, *probably*, *probably not*, or *definitely not* get vaccinated, plus those who were *unsure*
- *Pandemic severity*: pre-treatment averages of daily case and death counts

These covariates serve two purposes. In the frequentist specification, they enter the optimization that selects donor weights, helping ensure the synthetic Ohio resembles real Ohio along dimensions that plausibly drive vaccination behavior. In the Bayesian specification (Section IV), they appear as a second likelihood term that pulls the donor-weight posterior toward configurations under which $\mathbf{Z}_{\text{donors}}\beta \approx \mathbf{Z}_{\text{Ohio}}$, regularizing the estimation in addition to the horseshoe prior on the weights.

Mapping to the Potential Outcomes Framework

In the notation of Section I, Ohio is the single treated unit ($i = 0$) and we observe its post-treatment outcome Y_{0t} for $t > T_0$ — this is the realized trajectory under treatment, Y_1 . The donor states provide the raw material from which we estimate the missing Y_0 for Ohio: the vaccination rate Ohio would have had absent the lottery. The estimated daily treatment effect is

$$\hat{\alpha}_t = Y_{0t} - Y_{0t}^{\text{synth}} \quad \text{for } t > T_0,$$

and the time-averaged daily difference (TADD) over the post-treatment period,

$$\text{TADD} = \frac{1}{T - T_0} \sum_{t=T_0+1}^T \hat{\alpha}_t,$$

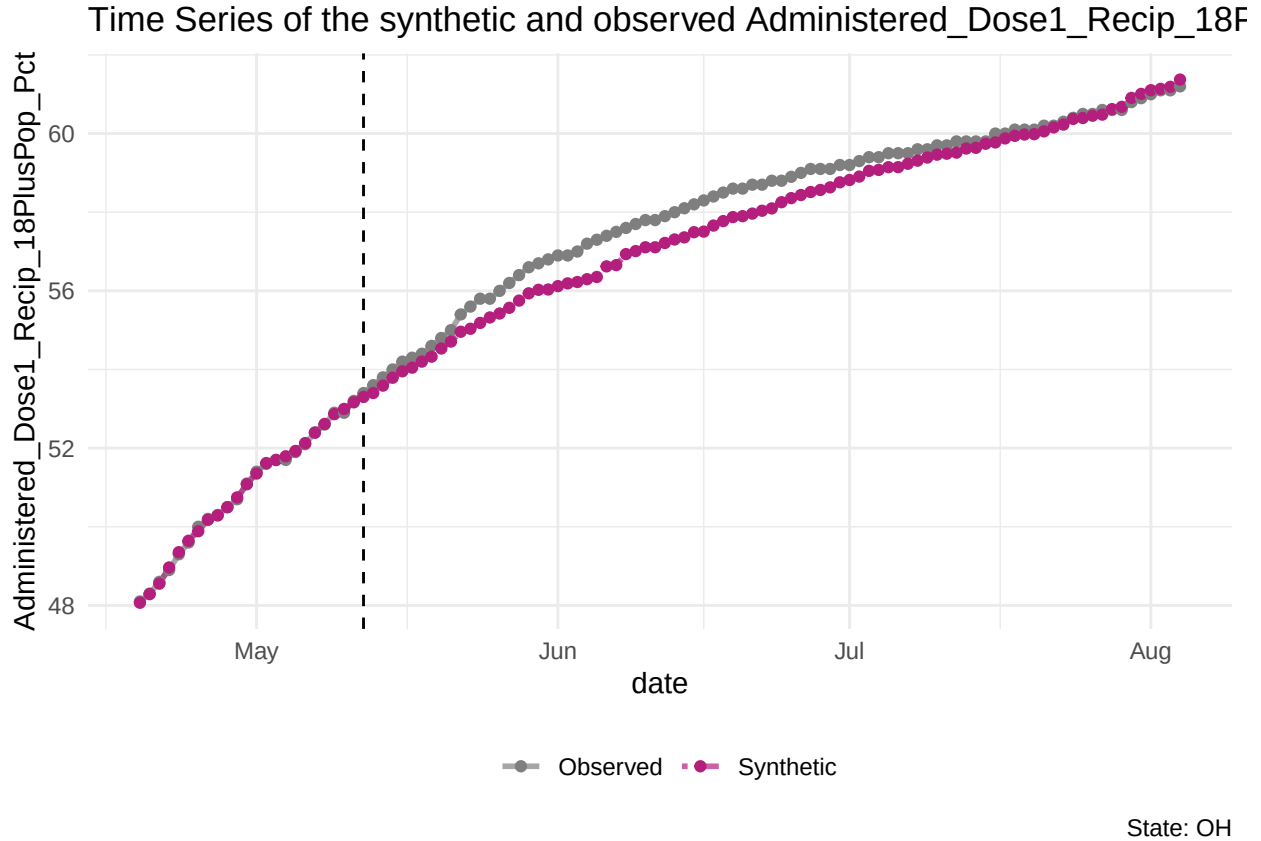
is our summary measure of the lottery’s effect on first-dose uptake. The remainder of the tutorial is concerned with two ways of estimating Y_{0t}^{synth} — first frequentist, then Bayesian — and with what each approach lets us say about the resulting causal estimate.

III. Frequentist Framework

Before moving to the Bayesian formulation, we decided to first implement a classical frequentist synthetic control. We wanted a concrete baseline to compare against — running the standard approach on our own data gives us a clear reference point and lets us show, rather than just describe, what the Bayesian framework adds. The contrast in how each approach handles uncertainty becomes much easier to see once both are applied to the same problem.

The plot below shows the trend in Ohio’s actual versus synthetic (counterfactual) vaccination rates before and after the lottery announcement on May 12, 2021

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    4.0.0      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## Loading required package: StanHeaders
##
##
## rstan version 2.32.7 (Stan version 2.32.2)
##
##
## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using `reduce_sum()` or `map_rect()` Stan functions,
## change `threads_per_chain` option:
## rstan_options(threads_per_chain = 1)
##
##
##
## Attaching package: 'rstan'
##
##
## The following object is masked from 'package:tidyr':
##
##      extract
```



Frequentist TADD: 0.3966

IV. Bayesian Inference

The Bayesian synthetic control method replaces the frequentist point-estimate framework with a fully probabilistic one. Instead of finding a single set of donor weights that minimizes a loss function, we place **priors** on the weights and learn a **posterior distribution** over them given the pre-treatment data. This gives us genuine uncertainty quantification — at every post-treatment time point, we get a distribution over the counterfactual, not just a single predicted value.

The key question the model answers is: *“Given what Ohio looked like before the lottery, and given what the donor states looked like both before and after, what would Ohio’s vaccination rate have looked like had there been no lottery?”* The difference between that counterfactual and the observed Ohio trajectory is the estimated treatment effect.

a. Dataset Formatting

We split the data into pre- and post-treatment periods: the pre-treatment donor trajectories are used to learn the counterfactual weights, and the post-treatment donor trajectories are used to project the synthetic Ohio counterfactual.

b. Bayesian Model Specification

The core of our Bayesian synthetic control is a **linear regression with a Horseshoe prior** on the donor weights. The model has two components working together:

Outcome model: Ohio’s pre-treatment vaccination rate is modeled as a weighted combination of donor states’ rates plus an intercept. In matrix form: $y_{\text{train}} \sim \mathcal{N}(\beta_0 + X_{\text{train}}\beta, \sigma^2)$, where $\beta \in \mathbb{R}^p$ are the donor weights we want to learn.

Covariate balance term: We add a second likelihood term that penalizes weights which produce a synthetic control with covariate profiles different from Ohio’s. This encodes the intuition that a good synthetic control should resemble Ohio not just in its outcome trajectory, but also in its structural characteristics (demographics, political lean, vaccine hesitancy). Formally: $z_{\text{treated}} \sim \mathcal{N}(Z_{\text{donors}}\beta, 1/\sqrt{\lambda_{\text{cov}}})$, where **covariate_weight** λ_{cov} controls how strongly we enforce this balance.

Why the Horseshoe prior? With 31 donor states, we expect only a few to be truly informative for constructing synthetic Ohio — most should get near-zero weight. The Horseshoe prior is designed exactly for this *sparse signal* setting. It uses a **global shrinkage parameter** τ that pulls all weights toward zero, and **local shrinkage parameters** λ_j that allow individual donor weights to escape shrinkage if the data strongly support them. This is more flexible than Lasso (which can over-shrink large signals) and more principled than Ridge (which shrinks everything equally regardless of relevance).

```
# =====
# BAYESIAN SYNTHETIC CONTROL: HORSESHOE PRIOR + COVARIATES
# =====

stan_code_full_cov = "
data {
  // Dimensions
  int<lower=1> N_train;      // Number of pre-treatment time points
  int<lower=1> N_test;      // Number of post-treatment time points
  int<lower=1> p;           // Number of donor states
  int<lower=1> K;           // Number of covariates

  // Outcome data
  vector[N_train] y_train;  // Ohio pre-treatment vaccination rates
  matrix[N_train, p] X_train; // Donor pre-treatment rates (design matrix)
  matrix[N_test, p] X_test;  // Donor post-treatment rates (for prediction)

  // Covariate balance data (standardized)
  vector[K] z_treated;      // Ohio's standardized covariate vector
  matrix[K, p] Z_donors;    // Donor states' standardized covariate matrix

  // Strength of covariate balance enforcement:
  // Higher values → synthetic control must match Ohio's covariates more closely
  real<lower=0> covariate_weight;
}

parameters {
  real beta_0;              // Intercept
  real<lower=0> sigma2;      // Variance of the outcome model
  vector[p] beta_raw;       // Raw (unscaled) donor weights
  vector<lower=0, upper=pi()/2>[p] lambda_unif; // Uniform auxiliary for local scales
  real<lower=0> tau;         // Global shrinkage: pulls ALL weights toward 0
}

transformed parameters {
  vector[p] beta;           // Final donor weights (after Horseshoe scaling)
  real<lower=0> sigma;       // Standard deviation (sqrt of sigma2)
  vector<lower=0>[p] lambda; // Local shrinkage scales - one per donor state
  vector[N_train] X_beta;   // Fitted values in the pre-treatment period
}
```

```

// Horseshoe reparameterization using the half-Cauchy via arctangent trick:
// lambda_unif ~ Uniform(0, pi/2) → lambda = tau * tan(lambda_unif)
// This is equivalent to lambda_j ~ Half-Cauchy(0, tau), but more numerically
// stable for HMC. Each beta[j] = lambda[j] * beta_raw[j].
// Interpretation: donors with large lambda[j] can have large weights;
// those with small lambda[j] are shrunk toward zero by tau.
lambda = tau * tan(lambda_unif);
for (j in 1:p) {
  beta[j] = lambda[j] * beta_raw[j];
}
sigma = sqrt(sigma2);
X_beta = beta_0 + X_train * beta;
}
model {
  // --- Priors ---

  // beta_raw ~ N(0,1): standard normal on raw weights before Horseshoe scaling.
  // The actual scale of beta[j] is determined by lambda[j], not beta_raw[j].
  beta_raw ~ normal(0, 1);

  // tau ~ Half-Cauchy(0, sigma): global shrinkage scale, data-adaptive.
  // Anchored to sigma so the amount of shrinkage scales with outcome variability.
  // If sigma is large (noisy data), tau is allowed to be large too.
  tau ~ cauchy(0, sigma);

  // Weakly informative priors on sigma and intercept.
  // Cauchy(0,10) is heavy-tailed - allows large values if the data support them
  // but regularizes toward zero. Appropriate when we have little prior knowledge
  // about the scale of the vaccination rate or the intercept.
  sigma ~ cauchy(0, 10);
  beta_0 ~ cauchy(0, 10);

  // --- Likelihood 1: Outcome model ---
  // Ohio's pre-treatment vaccination rates are modeled as a linear combination
  // of donor rates, weighted by beta, plus Gaussian noise.
  y_train ~ normal(X_beta, sigma);

  // --- Likelihood 2: Covariate balance term ---
  // We want Z_donors * beta z_treated, i.e., the weighted combination of
  // donor covariates should match Ohio's standardized covariate vector.
  // This acts as a soft constraint: covariate_weight controls its strength.
  // Think of it as adding a prior that says 'prefer weights that produce a
  // demographically similar synthetic Ohio.'
  z_treated ~ normal(Z_donors * beta, 1.0 / sqrt(covariate_weight));
}
generated quantities {
  // Post-treatment counterfactual: apply the learned donor weights to the
  // donors' post-treatment trajectories to predict what Ohio WOULD have looked
  // like without the lottery. The difference between this and actual Ohio
  // is our estimated treatment effect at each time point.
  vector[N_test] y_test_pred;
  for (i in 1:N_test) {
    y_test_pred[i] = beta_0 + X_test[i,] * beta;
  }
}

```

```

}
}
"

# Pass all required data to Stan.
# covariate_weight = 1.0 means moderate enforcement of covariate balance.
# Increasing this forces stricter demographic matching at the cost of outcome fit.
stan_data_full = list(
  N_train      = length(ohio_pre),
  N_test       = nrow(donor_post),
  p            = length(all_donors),
  K            = length(covariate_names),
  y_train      = ohio_pre,
  X_train      = donor_pre,
  X_test       = donor_post,
  z_treated    = ohio_cov_z,
  Z_donors     = donor_cov_z,
  covariate_weight = 1.0
)

# Run MCMC via Stan's HMC/NUTS sampler.
# iter = 4000 total per chain, warmup = 2000 discarded for adaptation.
# We retain 2000 posterior draws × 4 chains = 8000 total posterior samples.
# adapt_delta = 0.99: accept smaller HMC step sizes → fewer divergent transitions.
# This is especially important for the Horseshoe, whose posterior has a
# challenging funnel geometry near tau = 0.
# max_treedepth = 15: allows NUTS to build deeper trajectory trees for complex
# posteriors - at the cost of more computation per iteration.
fit_full = stan(
  model_code = stan_code_full_cov,
  data       = stan_data_full,
  iter       = 4000,
  warmup     = 2000,
  chains     = 4,
  seed       = 42,
  control    = list(adapt_delta = 0.99, max_treedepth = 15)
)

```

```
## Trying to compile a simple C file
```

c. Posterior Inference

After sampling, we have 8,000 posterior draws of every parameter — each draw is a complete, jointly plausible set of donor weights, intercept, and noise level. We summarize these draws to compute the **TADD (Time-Averaged Daily Difference)**, giving us a full posterior distribution over the average treatment effect rather than a single point estimate.

```
## Bayesian TADD (posterior mean): 0.3617
```

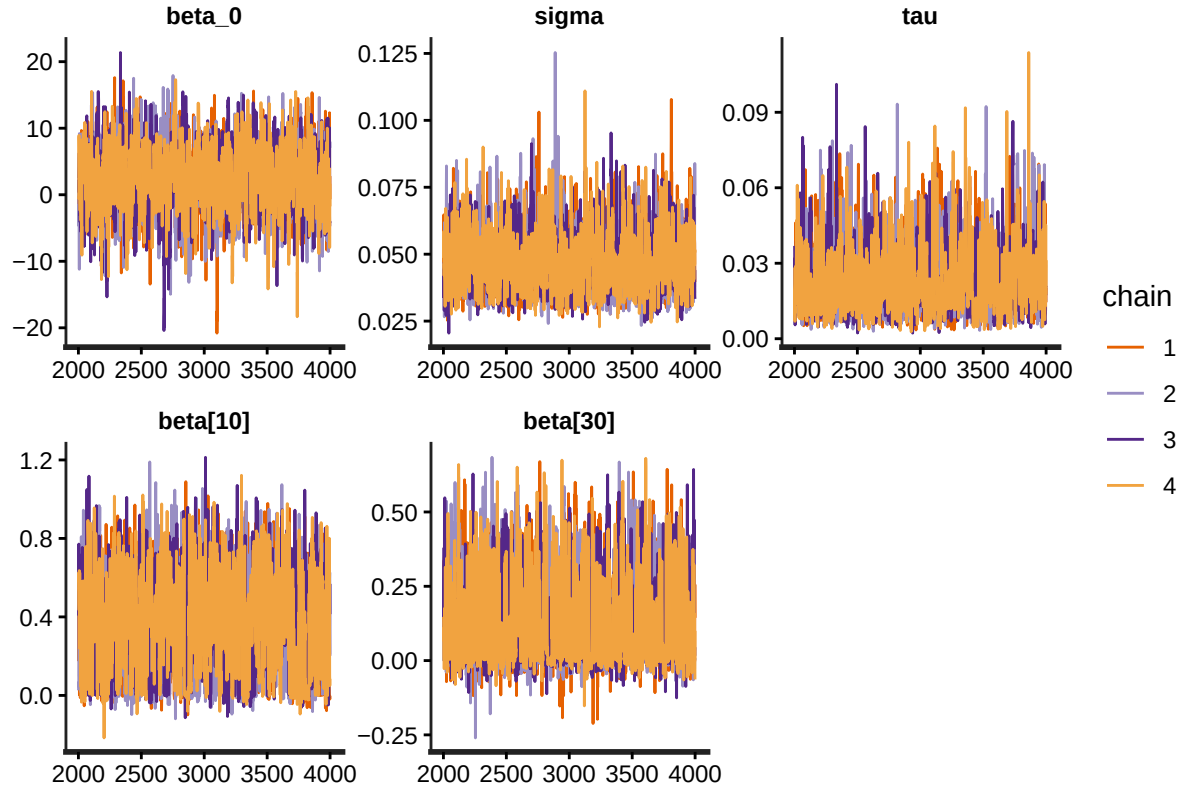
```
## 95% credible interval: [-0.369, 1.1424]
```

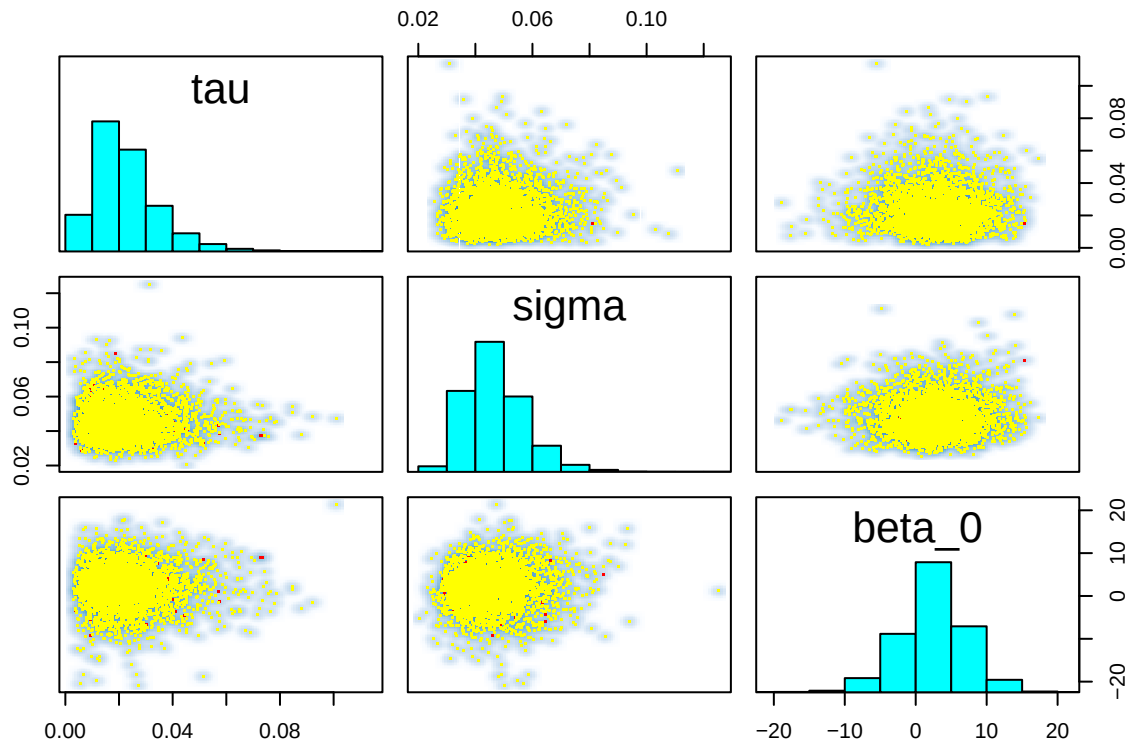
```
## P(TADD > 0): 0.8344
```

d. MCMC Diagnostics

Before interpreting any posterior summaries, we verify that the sampler converged. We check four diagnostics: **R-hat** (between-chain vs. within-chain variance; should be ≈ 1), **ESS** (effective independent draws after accounting for autocorrelation), **divergent transitions** (HMC integration failures in high-curvature regions), and **treedepth saturation** (NUTS hitting its trajectory limit).

##	Diagnostic	Value	Threshold	Status
##	R-hat (max)	1.005	< 1.01	Pass
##	R-hat > 1.01 (count)	0	= 0	Pass
##	R-hat > 1.05 (count)	0	= 0	Pass
##	ESS (minimum)	603	> 400	Pass
##	ESS (median)	1946	> 1000	Pass
##	ESS < 400 (count)	0	= 0	Pass
##	Divergent transitions	80 / 8000 (1.0%)	< 1%	Fail
##	Treedepth saturation (max = 15)	7920 / 8000 (99.0%)	< 5%	Inefficient





V. Results

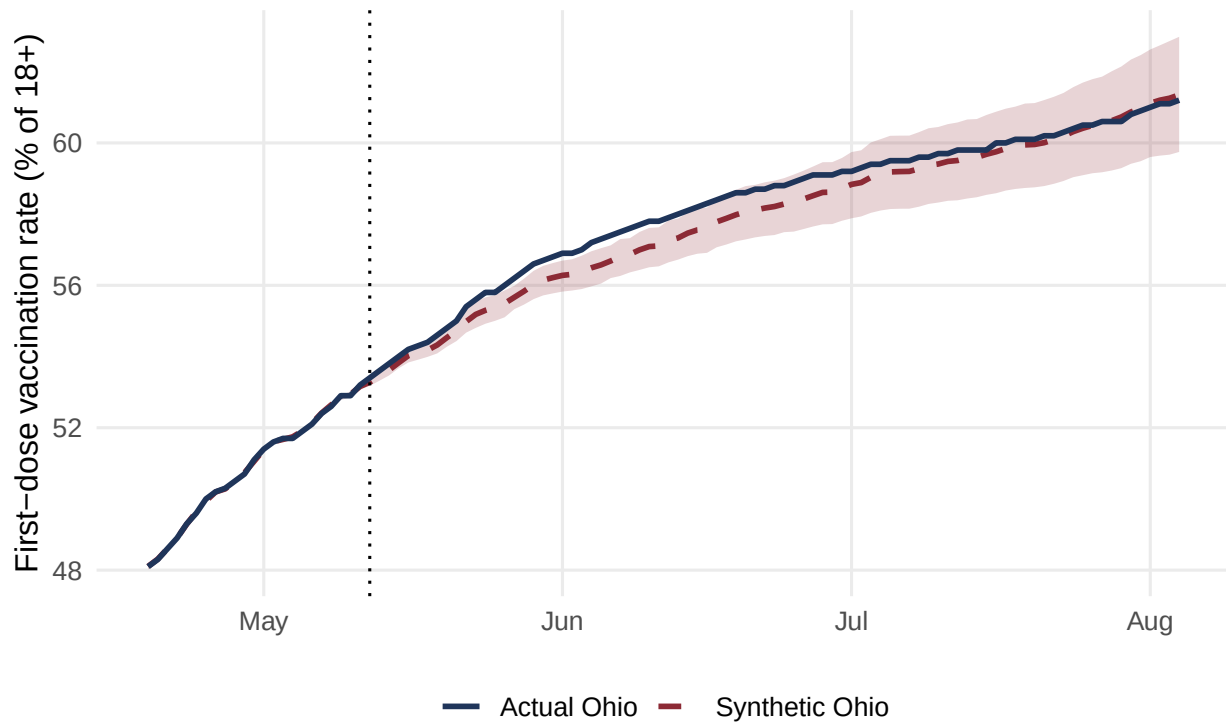
##	Model	Point Estimate (TADD)
## 1	Frequentist SCM	0.40
## 2	Bayesian (simple, 5 donors)	0.31
## 3	Bayesian (horseshoe, 31 donors, 13 covariates)	0.36

##	95% Interval	Inference	Advantages
## 1	-	Placebo test	Simple, fast, standard
## 2	[-0.14, 0.77]	P(effect > 0) = 90.4%	Uncertainty quantification
## 3	[-0.37, 1.14]	P(effect > 0) = 83.4%	Full regularization + uncertainty

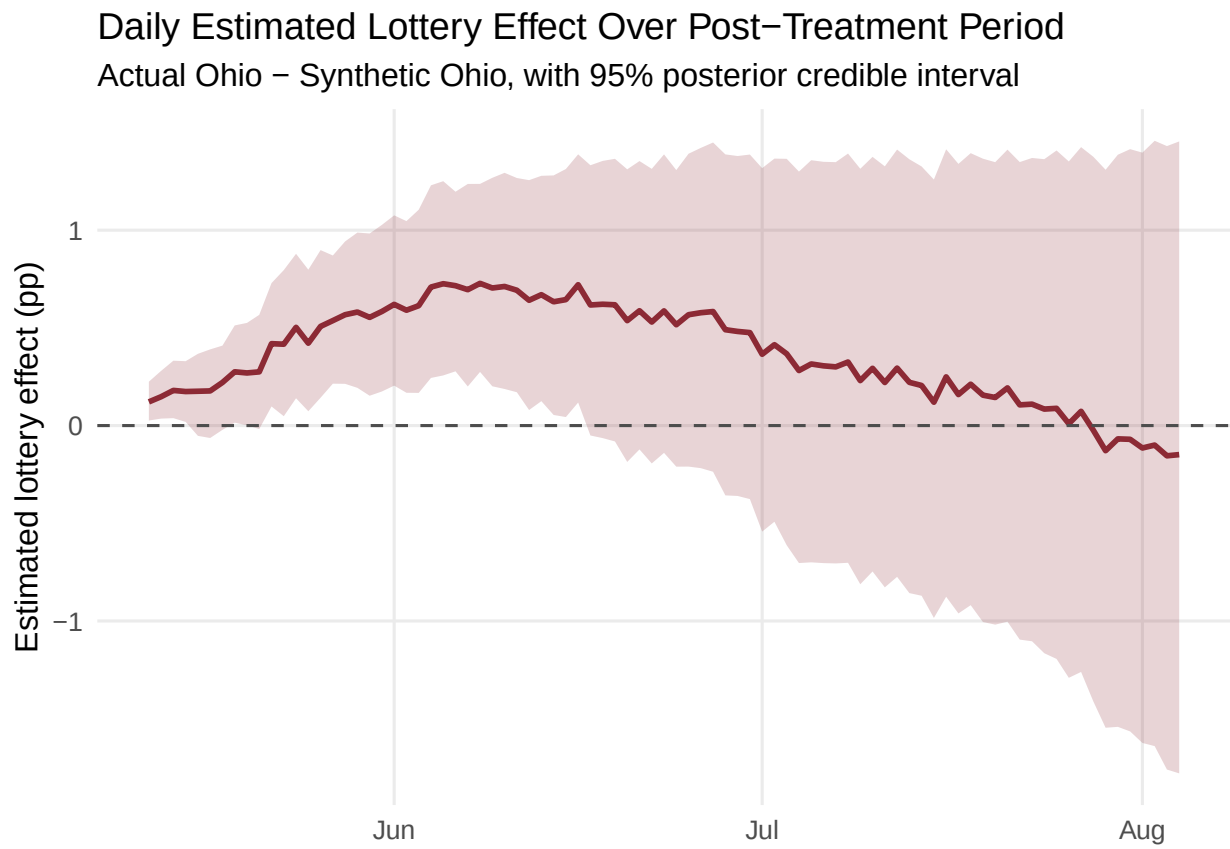
a. Synthetic Ohio Trajectory and Actual Trajectory Comparison

Actual vs. Synthetic Ohio

Shaded band = 95% posterior credible interval; dotted line = May 12, 2021



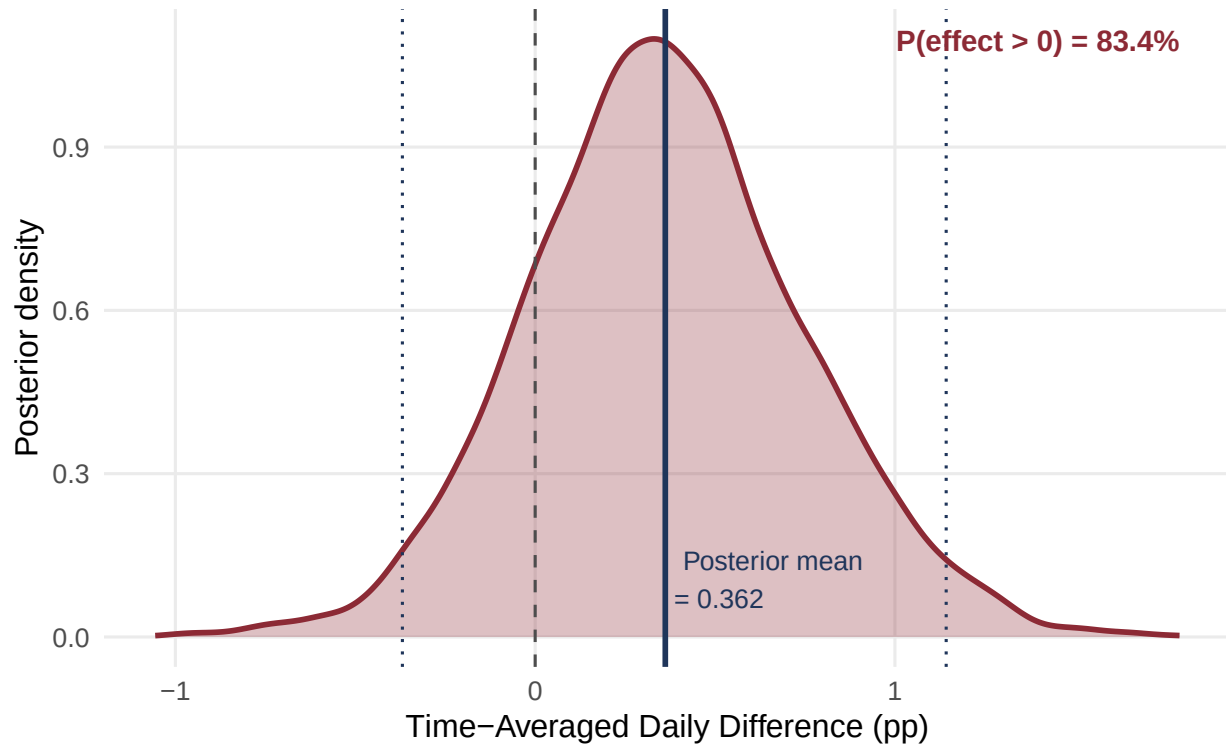
b. Daily Treatment Effect with Uncertainty



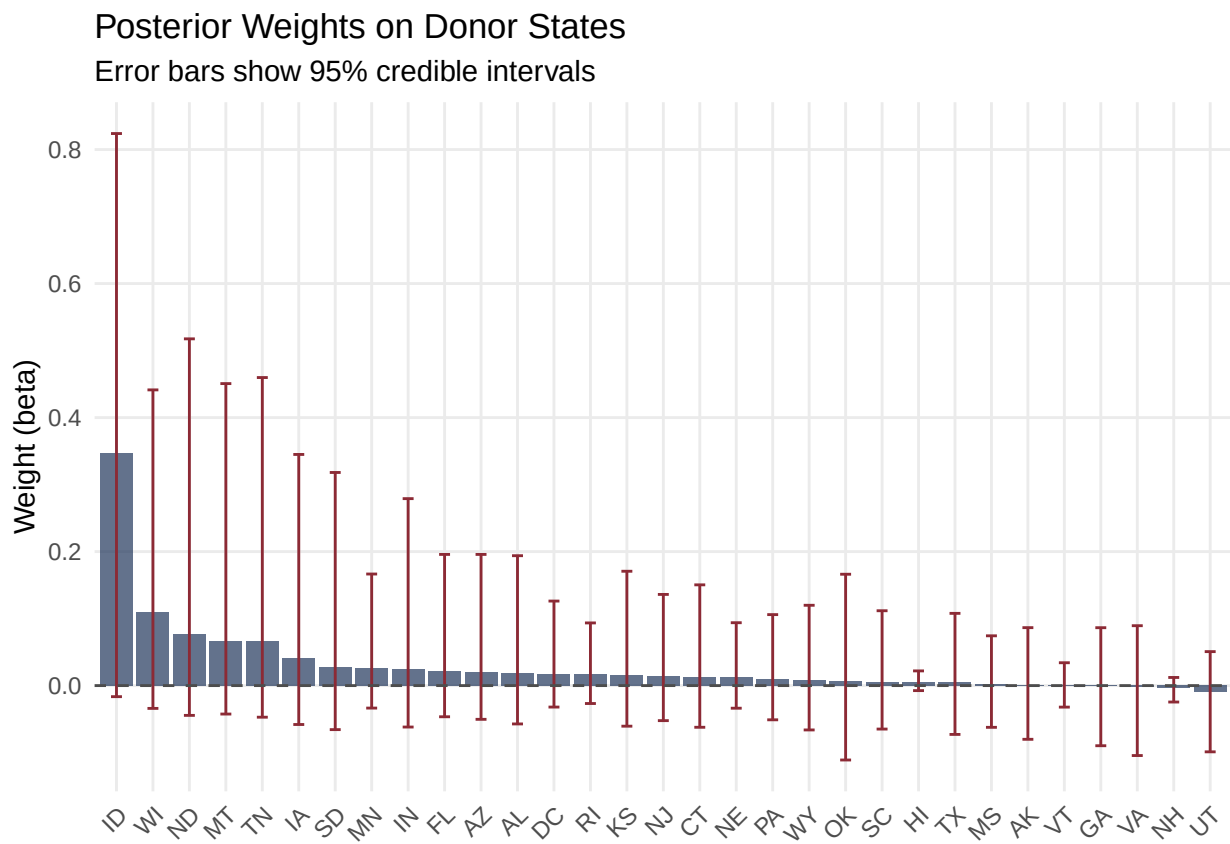
c. Posterior Distribution of TADD

Posterior Distribution of Ohio Vax-a-Million Lottery Effect

95% credible interval: [-0.369, 1.142]



d. Posterior Donor Weights



e. Comparing Frequentist Vs Bayesian Results

